Lecture 32: Interpreter Design in Scheme

By Pushpendra Uikey

1. Objective

Design a minimalist interpreter for a subset of the **Scheme** language, capable of evaluating expressions in a given environment.

2. Core Function: `eval`

```
(define (eval exp env)
  (cond ((number? exp) exp)
        ((eq? (car exp) 'quote) (cadr exp))
        ((symbol? exp) (lookup exp env))
        ((eq? (car exp) 'lambda) (list 'closure (cadr exp) (caddr
           ↪  exp) env))
        ((eq? (car exp) 'cond) (evalcond (cdr exp) env))
        (else (apply (eval (car exp) env)
                     (evallist (cdr exp) env)))))
```

Explanation

- **Constants:** Numbers evaluate to themselves.
- **Quote:** Returns the quoted expression without evaluation.
- **Symbols:** Values are looked up in the current environment.
- **Lambda:** Produces a **closure** in the form `(closure params body env)`.
- **Cond:** Delegated to the helper `evalcond`.
- **Application:** The operator and operands are evaluated, then passed to `apply`.

Each operand is evaluated independently (no side-effect interference). Closures preserve the lexical environment where they are defined.

3. Procedure Application: `apply`

```
(define (apply proc args)
  (cond ((primitive? proc) (apply-prim proc args))
        (else (eval (caddr proc)
                    (extend-env (cadddr proc) (cadr proc) args)))
                      ↪ ))
```

Explanation

- If `proc` is primitive (e.g. `+`, `car`), delegate to `apply-prim`.
- Otherwise, it is a closure. Evaluate its body in an **extended environment**.
- The extended environment binds parameters (`(cadr proc)`) to the argument values.

4. Conditional Evaluation: `evalcond`

```
(define (evalcond clauses env)
  (cond ((eq? clauses '()) '())
        ((eq? (caar clauses) 'else) (eval (cadar clauses) env))
        ((false? (eval (caar clauses) env)) (evalcond (cdr
           ↪ clauses) env))
        (else (eval (cadar clauses) env))))
```

Explanation

Evaluates each clause in sequence:

1. If no clauses remain, return the empty list.
2. If the first clause is `else`, evaluate its body.
3. If the test evaluates to `false`, proceed to the next clause.
4. Otherwise, evaluate the associated expression.

5. Supporting Concepts

- **Environment (env):** Association list of variable bindings.
- **Closure:** A combination of parameters, body, and environment:

$$\text{closure} = (params, body, env)$$

- `evallist`**:** Evaluates all arguments in order.
- `lookup`**:** Searches a variable's value in the environment.
- `apply-prim`**:** Applies host-language primitives.

6. Evaluation Model Summary

For an expression $(f\, a_1\, a_2 \ldots a_n)$:

1. Evaluate the operator f to obtain a procedure (primitive or closure).
2. Evaluate each operand a_i to its argument value.
3. Apply the procedure using `apply`.

7. Remarks

- The interpreter enforces **lexical scoping** via environment capture.
- It demonstrates the **metacircular evaluation model**—a Scheme interpreter written in Scheme.
- The recursive `eval{apply` cycle mirrors the real Scheme execution process.

8. Summary Diagram (Conceptual)

Expression $\xrightarrow{\text{eval}}$ Value

$(f\ a_1\ a_2 \ldots a_n) \xrightarrow{\text{eval}}$ (Apply f to args)

If closure: evaluate body in extended env